

Question No:8

Question description

Selvan is very interested in surfing the contents from google. He searches for various coding test on Google.

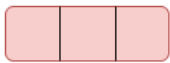
One day he searched about online coding competitions, in the retrieval links, he received many links for coding competition. he chooses first link from the goole suggestion list.

first question for the coding competition is LRU cache implementation using queue concepts.

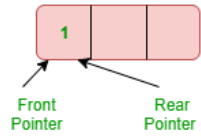
Function Description

1. **Queue** which is implemented using a doubly linked list. The maximum size of the queue will be equal to the total number of frames available (cache size). The most recently used pages will be near front end and least recently pages will be near the rear end.
2. **A Hash** with page number as key and address of the corresponding queue node as value.

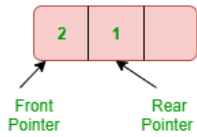
Given 3 page frames, so we take size of Queue is 3. Initially, Queue is empty.



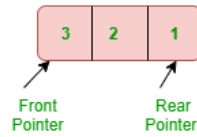
Input : 1



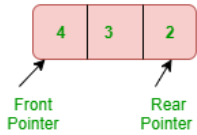
Input : 2 (every new input will be front as defined LRU)



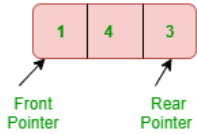
Input : 3



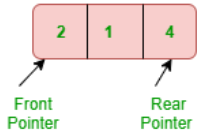
Input : 4 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



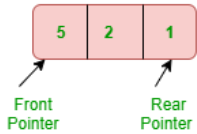
Input : 1 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



Input : 2 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



Input : 5 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



Constraints

For this experiment, cache can hold 4 pages.

Let 10 different pages can be requested (pages to be referenced are numbered from 0 to 9).

Input Format:

First line represents n and m, where n is the page number with in the range (0-9) & m is cache size (must be 4 for this problem).

Next line represents the reference pages.

Output Format:

Single line represents the cache frames after the above referenced pages.



Test Case 1

INPUT (STDIN)

```
6 4
1 2 3 1 4 5
```

EXPECTED OUTPUT

```
5 4 1 3
```

Test Case 2

INPUT (STDIN)

```
10 4
3 1 4 5 9 8 7 6 1 2
```

EXPECTED OUTPUT

```
2 1 6 7
```



Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node
5  {
6      int data;
7      struct Node *next;
8  };
9
10 struct Node *front = NULL;
11 struct Node *rear = NULL;
12
13 int size = 0;
14 int capacity;
15
16 /* Check whether page already exists */
17 struct Node *findNode(int page)
18 {
19     struct Node *temp = front;
20
21     while (temp != NULL)
22     {
23         if (temp->data == page)
24             return temp;
25
26         temp = temp->next;
27     }
28
29     return NULL;
```

```
30     }
31
32     /* Remove a page from the queue */
33     void removePage(int page)
34     {
35         struct Node *temp = front;
36         struct Node *prev = NULL;
37
38         while (temp != NULL)
39         {
40             if (temp->data == page)
41             {
42                 if (prev == NULL)
43                 {
44                     front = temp->next;
45                 }
46                 else
47                 {
48                     prev->next = temp->next;
49                 }
50
51                 if (temp == rear)
52                 {
53                     rear = prev;
54                 }
55             }
56         }
57     }
58 }
```

```
55
56     free(temp);
57     size--;
58
59     return;
60 }
61
62     prev = temp;
63     temp = temp->next;
64 }
65 }
66
67 /* Insert page at the front */
68 void insertFront(int page)
69 {
70     struct Node *newNode;
71     newNode = (struct Node *)malloc(sizeof(struct Node));
72     newNode->data = page;
73     newNode->next = front;
74
75     front = newNode;
76
77     if (rear == NULL)
78     {
```

```
79         rear = newNode;
80     }
81
82     size++;
83 }
84 /* LRU operation */
85 void accessPage(int page)
86 {
87     /* If page already exists */
88     if (findNode(page) != NULL)
89     {
90         removePage(page);
91         insertFront(page);
92     }
93     else
94     {
95         /* If queue is full, remove least recently used */
96         if (size == capacity)
97         {
98             int oldPage = rear->data;
99             removePage(oldPage);
100         }
101
102         insertFront(page);
103     }
104 }
```

```
106  /* Display queue */
107  void display()
108  {
109      struct Node *temp = front;
110
111      while (temp != NULL)
112      {
113          printf("%d", temp->data);
114
115          if (temp->next != NULL)
116              printf(" ");
117
118          temp = temp->next;
119      }
120
121      printf("\n");
122  }
123
124  int main()
125  {
126      int n;
127      int page;
128
129      scanf("%d %d", &n, &capacity);
```

```
130
131     for (int i = 0; i < n; i++)
132     {
133         scanf("%d", &page);
134         accessPage(page);
135     }
136     display();
137
138     return 0;
139 }
```

Output:

Test Case-1:

	6	4	
1	2	3	1 4 5
5	4	1	3

Test Case-2:

	10	4	
3	1	4 5 9 8 7 6 1 2	
2	1	6 7	

Keshav Kumar Rauniyar
CH.SC.U4CSE25236